

Πανεπιστήμιο Δυτικής Αττικής
Σχολή Μηχανικών
Τμήμα Μηχανικών Πληροφορικής και Υπολογιστών

Ασφάλεια στην τεχνολογία της πληροφορίας

2η Εργασία

ΝΙΚΟΛΑΟΣ ΘΩΜΑΣ
ΑΜ: 21390068

ΑΘΗΝΑ
ΣΑΒΒΑΤΟ 5 ΑΠΡΙΛΙΟΥ 2025

Περιεχόμενα

Δραστηριότητα 1: Δημιουργία ιδιωτικού κλειδιού 3
Δραστηριότητα 2: Κρυπτογράφηση μηνύματος 4
Δραστηριότητα 3: Αποκρυπτογράφηση μηνύματος 6
Δραστηριότητα 4: Υπογραφή μηνύματος 7
Δραστηριότητα 5: Επαλήθευση Υπογραφής 8
Περίπτωση Α: 8
Περίπτωση Β: 10
Δραστηριότητα 6: Μη αυτόματη επαλήθευση πιστοποιητικού X.509 12
Εκτυπώσεις προγραμμάτων. 16
Πιστοποιητικά. 16
Αρχεία εργασίας: 18

Δραστηριότητα 1: Δημιουργία ιδιωτικού κλειδιού

```
p = F7E75FDC469067FFDC4E847C51F452DF
q = E85CED54AF57E53E092113E62F436F4F
e = 0D88C3
```

Εικόνα 1: Τιμές των p,q και e.

Για την δημιουργία του ιδιωτικού κλειδιού, υπολογίζουμε το N = p×q:

- Υπολογίζουμε το p − 1
- Υπολογίζουμε το q − 1
- Υπολογίζουμε το φ(N) = (p − 1)×(q − 1)
- Υπολογίζουμε τον αντίστροφο (mod_inverse) του e στο mod ϕ(N) και έτσι έχουμε το d.

Για την υλοποίηση των παραπάνω βημάτων χρησιμοποιούμε τις συναρτήσεις τις βιβλιοθήκης bn.

private_key.c

```c
#include <stdio.h>
#include <openssl/bn.h>

#define NBITS 256

void printBN(char *msg, BIGNUM * a){

    char * number_str = BN_bn2hex(a);
    printf("%s %s\n", msg, number_str);
    OPENSSL_free(number_str);
}

int main(){
    BN_CTX *ctx = BN_CTX_new();
    BIGNUM *p = BN_new();
    BIGNUM *q = BN_new();
    BIGNUM *e = BN_new();
    BIGNUM *t = BN_new();
    BIGNUM *n = BN_new();
    BIGNUM *subp1 = BN_new();
    BIGNUM *subq1 = BN_new();
    BIGNUM *fn = BN_new();
    BIGNUM *d = BN_new();

    BN_hex2bn(&p, "F7E75FDC469067FFDC4E847C51F452DF");
    BN_hex2bn(&q, "E85CED54AF57E53E092113E62F436F4F");
    BN_hex2bn(&e, "0D88C3");
    BN_hex2bn(&t, "1");

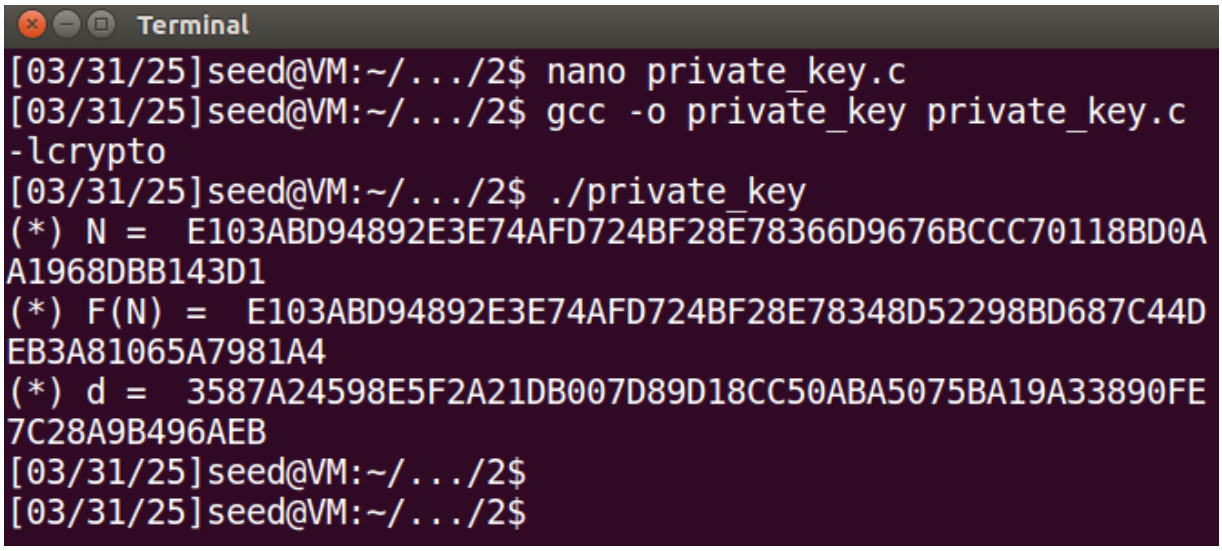
    BN_mul(n, p, q, ctx);
    BN_sub(subp1, p, t);
    BN_sub(subq1, q, t);
    BN_mul(fn, subp1, subq1, ctx);
    BN_mod_inverse(d, e, fn, ctx);

    printBN("(*) N = ", n);
    printBN("(*) F(N) = ", fn);
    printBN("(*) d = ", d);

}
```

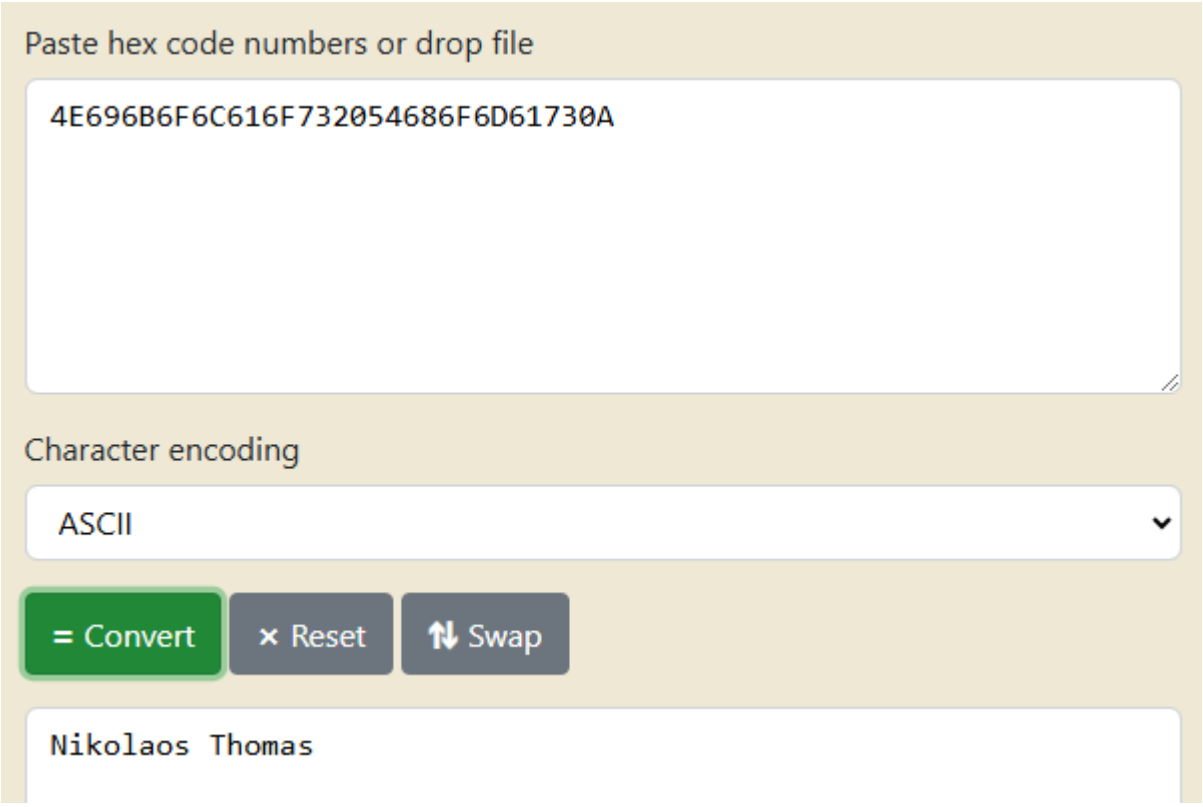
Line 11, Column 2 | Tab Size: 4 | C

Εικόνα 2: Κώδικας για την δημιουργία του ιδιωτικού κλειδιού.



Εικόνα 3: Μεταγλώττιση και εκτέλεση του προγράμματος.

Δραστηριότητα 2: Κρυπτογράφηση μηνύματος



Εικόνα 4: Μετατροπή string σε hex.

Για την κρυπτογράφηση του μηνύματος, θα χρησιμοποιήσουμε την συνάρτηση BN_mod_exp() της βιβλιοθήκης bn με το αρχικό μήνυμα.

encryption.c

```c
#include <stdio.h>
#include <openssl/bn.h>

#define NBITS 256

void printBN(char *msg, BIGNUM * a){

    char * number_str = BN_bn2hex(a);
    printf("%s %s\n", msg, number_str);
    OPENSSL_free(number_str);
}

int main(){

    BN_CTX *ctx = BN_CTX_new();
    BIGNUM *name = BN_new();
    BIGNUM *e = BN_new();
    BIGNUM *n = BN_new();
    BIGNUM *c = BN_new();
    BN_hex2bn(&name, "4E696B6F6C616F732054686F6D6173");
    BN_hex2bn(&e, "0D88C3");
    BN_hex2bn(&n, "E103ABD94892E3E74AFD724BF28E78366D9676BCCC70118BD0AA1968DBB143D1");

    BN_mod_exp(c, name, e, n, ctx);

    printBN("(*) Encrypted Message = ", c);
}

```

Εικόνα 5: Κώδικας για την κρυπτογράφηση μυνήματος.

Terminal

```
[03/31/25]seed@VM:~/.../2$ nano encryption.c
[03/31/25]seed@VM:~/.../2$ gcc -o encryption encryption.c -l
crypto
[03/31/25]seed@VM:~/.../2$ ./encryption
(*) Encrypted Message =  617AB29ECDA432F9B7C5F8E8044404EBFD8
B285A699ABED2575B252F78057164
[03/31/25]seed@VM:~/.../2$
```

Εικόνα 6: Μεταγλώττιση και εκτέλεση του προγράμματος.

Το πρόγραμμα εκτύπωσε το κρυπτογραφημένο μήνυμα.

Δραστηριότητα 3: Αποκρυπτογράφηση μηνύματος

Για την κρυπτογράφηση του μηνύματος, θα χρησιμοποιήσουμε την συνάρτηση BN_mod_exp() της βιβλιοθήκης bn με το κρυπτογραφημένο μήνυμα.

encryption.c | decryption.c

```c
#include <stdio.h>
#include <openssl/bn.h>

#define NBITS 256

void printBN(char *msg, BIGNUM * a){

    char * number_str = BN_bn2hex(a);
    printf("%s %s\n", msg, number_str);
    OPENSSL_free(number_str);
}

int main(){

    BN_CTX *ctx = BN_CTX_new();
    BIGNUM *d = BN_new();
    BIGNUM *n = BN_new();
    BIGNUM *c = BN_new();
    BIGNUM *m = BN_new();
    BIGNUM *start_m = BN_new();
    BN_hex2bn(&d, "3587A24598E5F2A21DB007D89D18CC50ABA5075BA19A33890FE7C28A9B496AEB");
    BN_hex2bn(&c, "617AB29ECDA432F9B7C5F8E8044404EBFD8B285A699ABED2575B252F78057164");
    BN_hex2bn(&n, "E103ABD94892E3E74AFD724BF28E78366D9676BCCC70118BD0AA1968DBB143D1");
    BN_hex2bn(&start_m, "4E696B6F6C616F732054686F6D6173");

    BN_mod_exp(m, c, d, n, ctx);

    printBN("(*) Decrypted Message = ", m);
    if(BN_cmp(start_m,m) == 0){
        printf("Messages are the same!\n");
    }else{
        printf("Messages are not the same!\n");
    }
}
```

Εικόνα 7: Κώδικας για την αποκρυπτογράφηση μυνήματος.

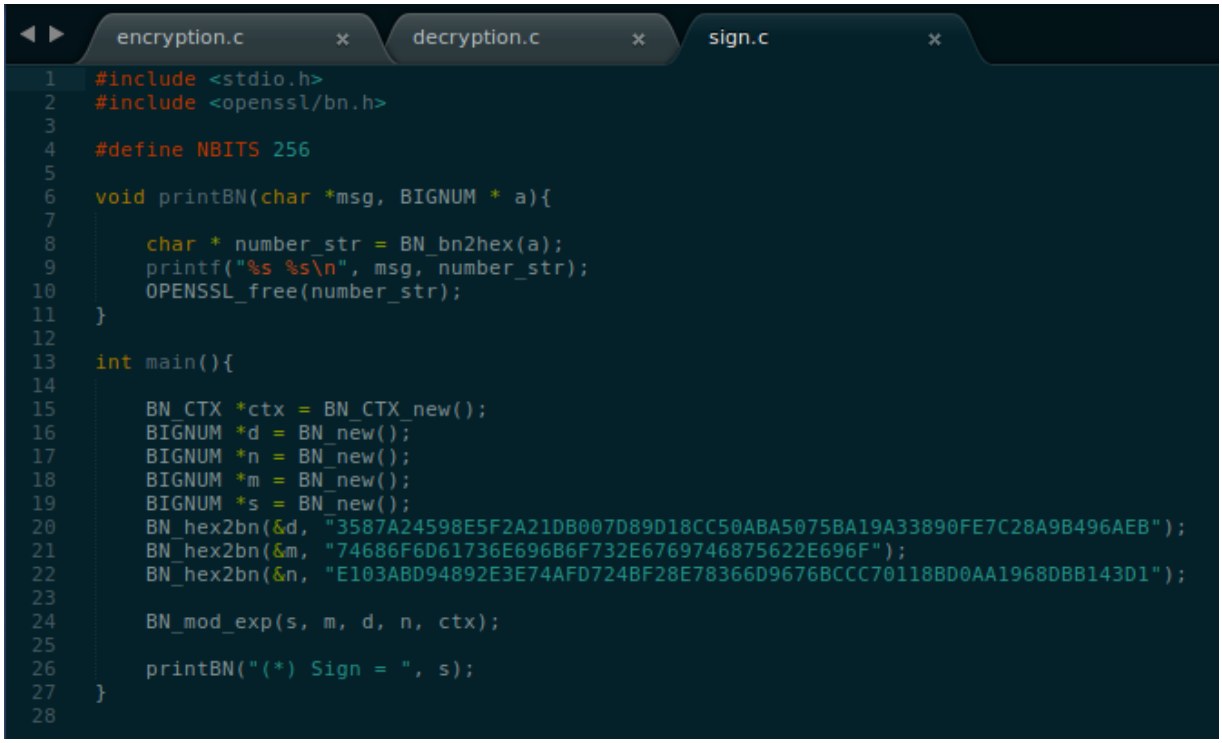
```
Terminal
[03/31/25]seed@VM:~/.../2$ nano decryption.c
[03/31/25]seed@VM:~/.../2$ gcc -o decryption decryption.c -l
crypto
[03/31/25]seed@VM:~/.../2$ ./decryption
(*) Decrypted Message =  4E696B6F6C616F732054686F6D6173
Messages are the same!
[03/31/25]seed@VM:~/.../2$
```

Εικόνα 8: Μεταγλώττιση και εκτέλεση του προγράμματος.

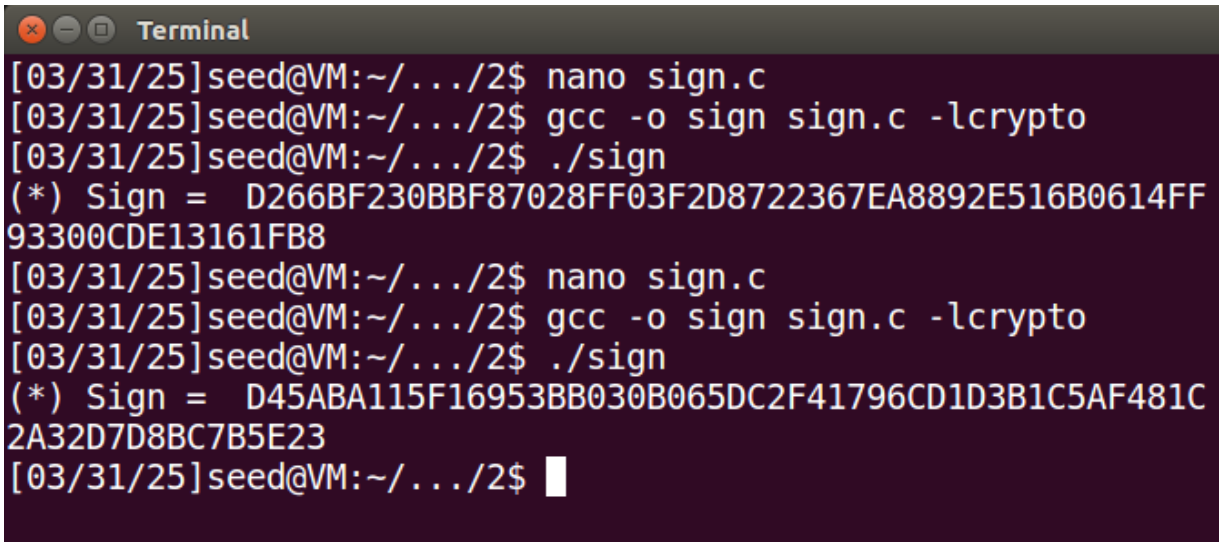
Παρατηρούμε ότι το πρόγραμμα εκτύπωσε το αρχικό μήνυμα.

Δραστηριότητα 4: Υπογραφή μηνύματος

Για την υπογραφή του μηνύματος θα χρησιμοποιήσουμε την συνάρτηση BN_mod_exp() της βιβλιοθήκης bn με το μήνυμα και το ιδιωτικό μας κλειδί.



Εικόνα 9: Κώδικας για την υπογραφή μηνύματος.



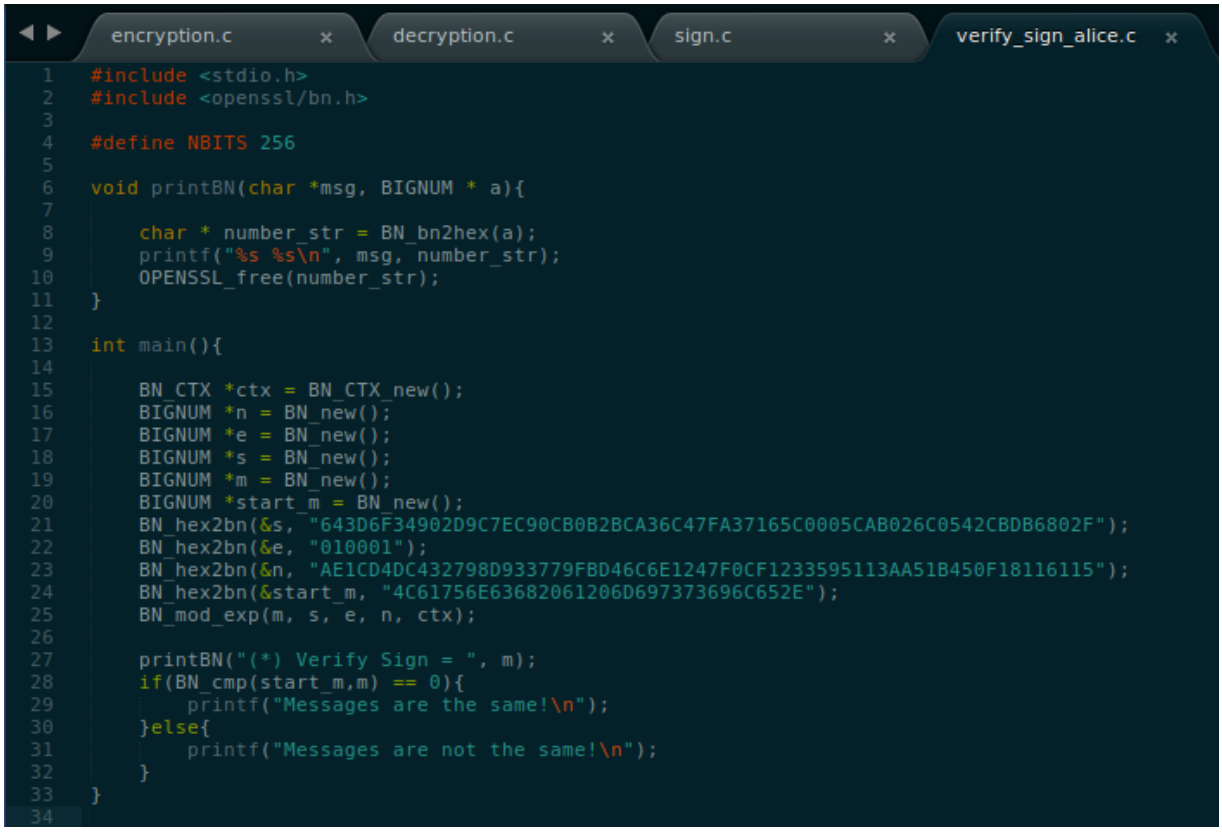
Εικόνα 10: Μεταγλώττιση και εκτέλεση του προγράμματος.

Όπως φαίνεται, και με μία πολύ μικρή αλλαγή σε ένα γράμμα του μηνύματος, το υπογεγραμμένο μήνυμα αλλάζει εντελώς.

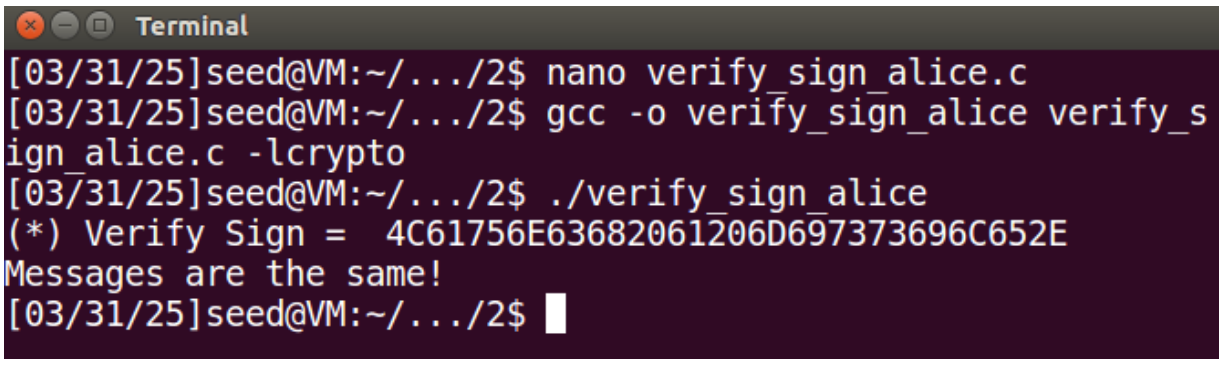
Δραστηριότητα 5: Επαλήθευση Υπογραφής

Περίπτωση Α:

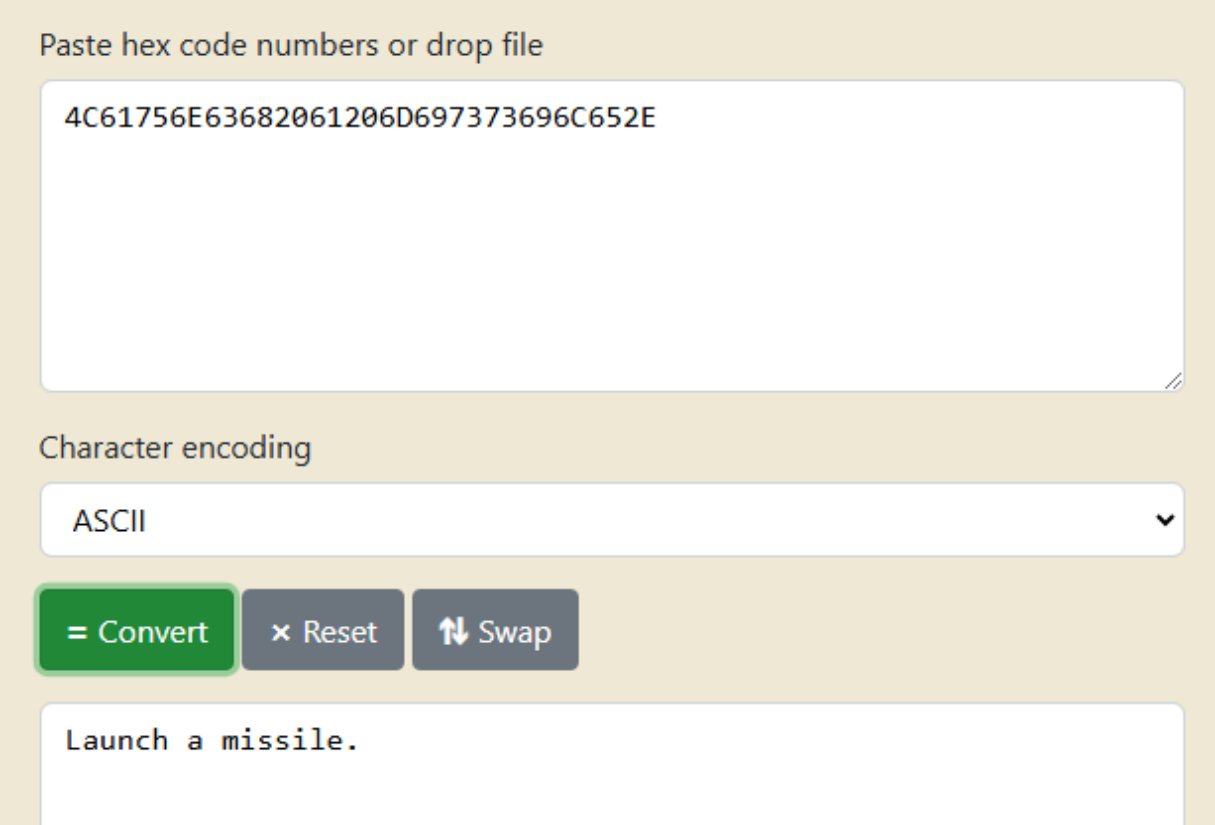
Για την επαλήθευση της υπογραφής θα χρησιμοποιήσουμε ξανά την συνάρτηση BN_mod_exp() της βιβλιοθήκης bn αλλά με διαφορά το δημόσιο κλειδί του αποστολέα.



Εικόνα 11: Επαλήθευση υπογραφής της Alice.



Εικόνα 12: Μεταγλώττιση και εκτέλεση του προγράμματος.



Εικόνα 13: Απόδειξη επαλήθευσης.

Αν υποθέσουμε τώρα ότι η υπογραφή έχει αλλοιωθεί (καταστραφεί), έτσι ώστε το τελευταίο byte της υπογραφής να αλλάζει από 2F σε 3F, το αποτέλεσμα του προγράμματος θα είναι εντελώς διαφορετικό από το προηγούμενο και δεν θα επαληθεύεται με το αρχικό.

```
[03/31/25]seed@VM:~/.../2$ nano verify_sign_alice.c
[03/31/25]seed@VM:~/.../2$ gcc -o verify_sign_alice verify_s
ign_alice.c -lcrypto
[03/31/25]seed@VM:~/.../2$ ./verify_sign_alice
(*) Verify Sign =  91471927C80DF1E42C154FB4638CE8BC726D3D66C
83A4EB6B7BE0203B41AC294
Messages are not the same!
[03/31/25]seed@VM:~/.../2$
```

Εικόνα 14: Μεταγλώττιση και εκτέλεση του προγράμματος.



Εικόνα 15: Απόδειξη επαλήθευσης.

Περίπτωση Β:

Για την επαλήθευση της υπογραφής θα χρησιμοποιήσουμε ξανά την συνάρτηση BN_mod_exp() της βιβλιοθήκης bn αλλά με διαφορά το δημόσιο κλειδί του αποστολέα.

encryption.c | decryption.c | sign.c | verify_sign_alice.c | verify_sign_bob.c

```c
#include <stdio.h>
#include <openssl/bn.h>

#define NBITS 256

void printBN(char *msg, BIGNUM * a){

    char * number_str = BN_bn2hex(a);
    printf("%s %s\n", msg, number_str);
    OPENSSL_free(number_str);
}

int main(){

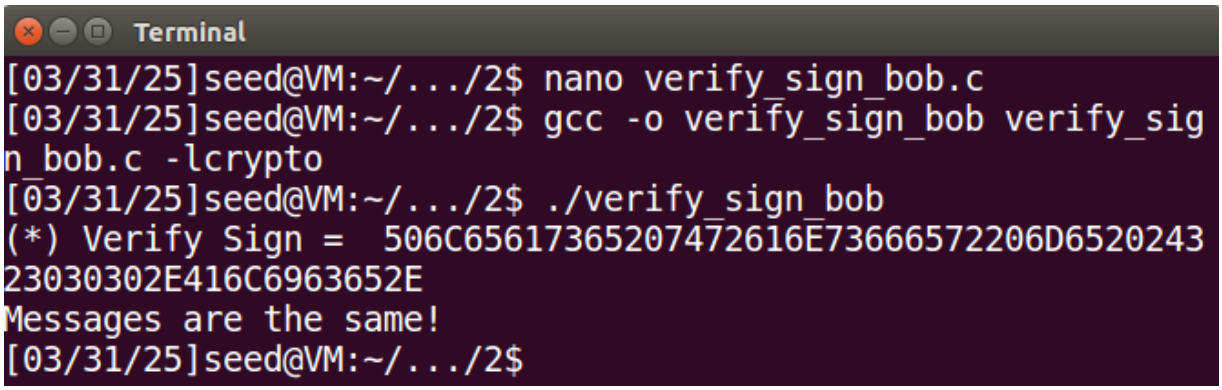
    BN_CTX *ctx = BN_CTX_new();
    BIGNUM *n = BN_new();
    BIGNUM *e = BN_new();
    BIGNUM *s = BN_new();
    BIGNUM *m = BN_new();
    BIGNUM *start_m = BN_new();

    BN_hex2bn(&s, "DB3F7CDB93483FC1E70E4EACA650E3C6505A3E5F49EA6EDF3E95E9A7C6C7A320");
    BN_hex2bn(&e, "010001");
    BN_hex2bn(&n, "DCBFFE3E51F62E09CE7032E2677A78946A849DC4CDDE3A4D0CB81629242FB1A5");
    BN_hex2bn(&start_m, "506C6561736520747261616E73666572206D652024323030302E416C6963652E");

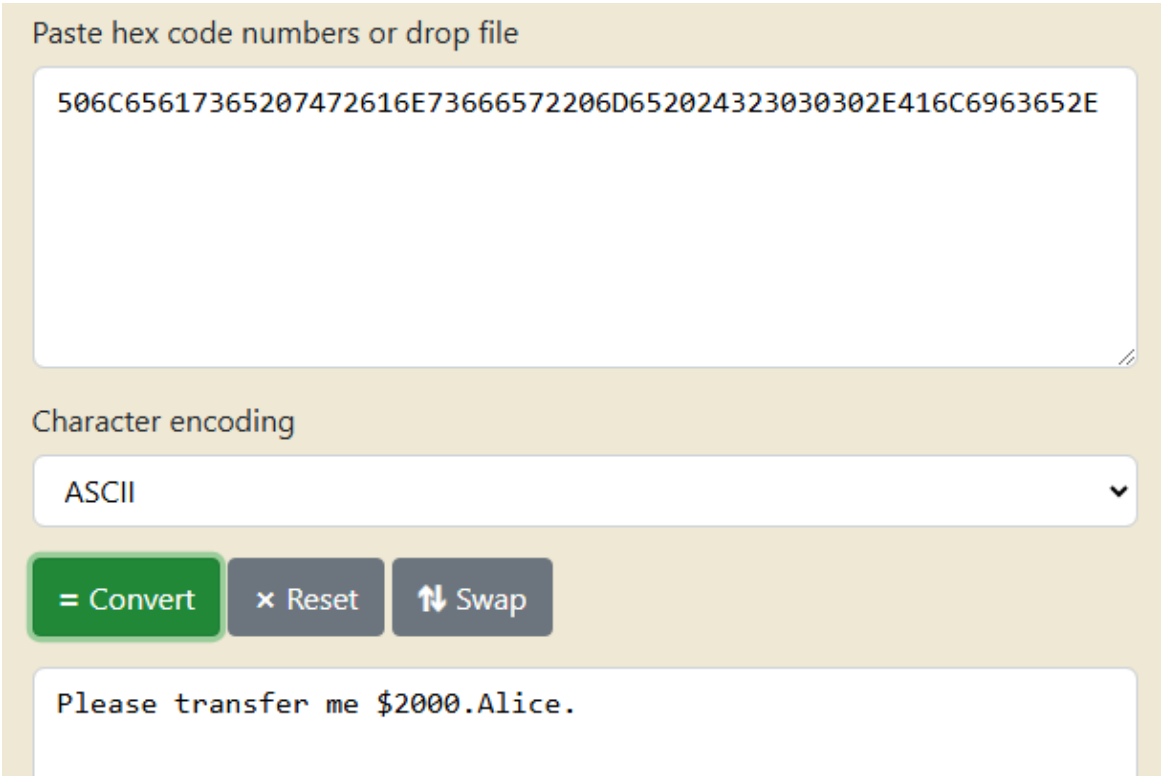
    BN_mod_exp(m, s, e, n, ctx);

    printBN("(*) Verify Sign = ", m);
    if(BN_cmp(start_m,m) == 0){
        printf("Messages are the same!\n");
    }else{
        printf("Messages are not the same!\n");
    }
}
```

Εικόνα 16: Επαλήθευση υπογραφής του Bob.

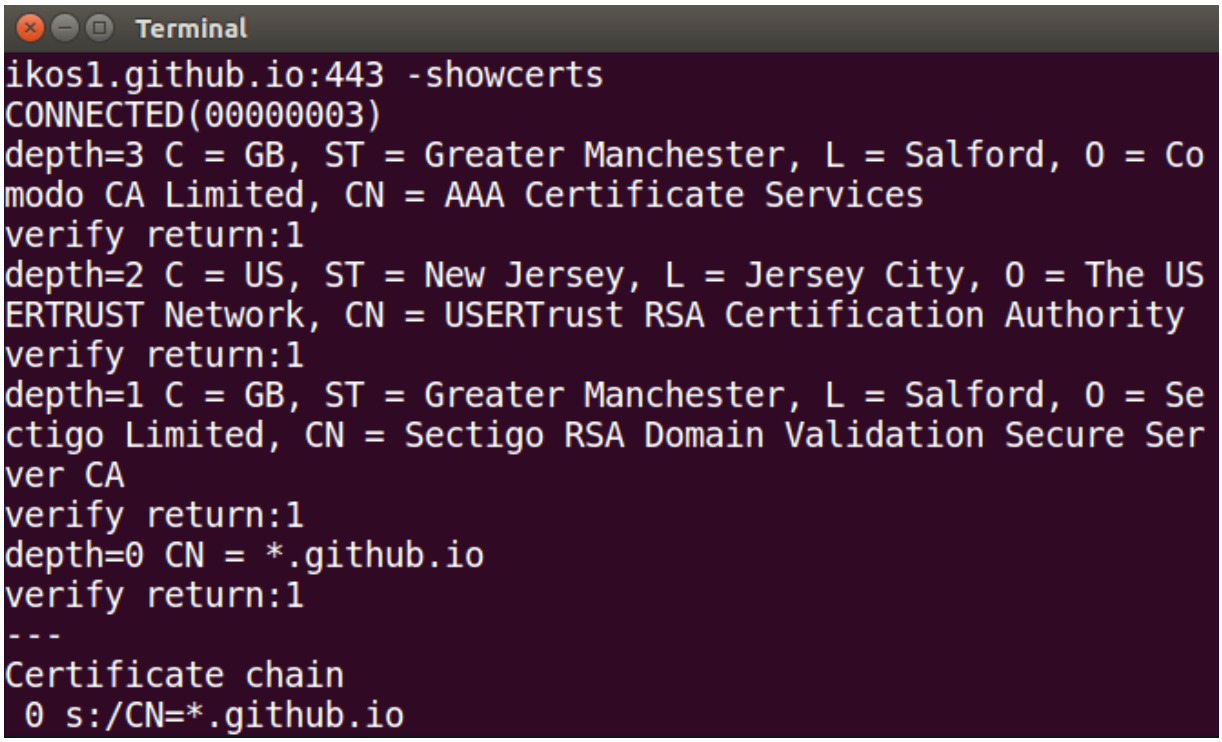


Εικόνα 17: Μεταγλώττιση και εκτέλεση του προγράμματος.



Εικόνα 18: Απόδειξη επαλήθευσης.

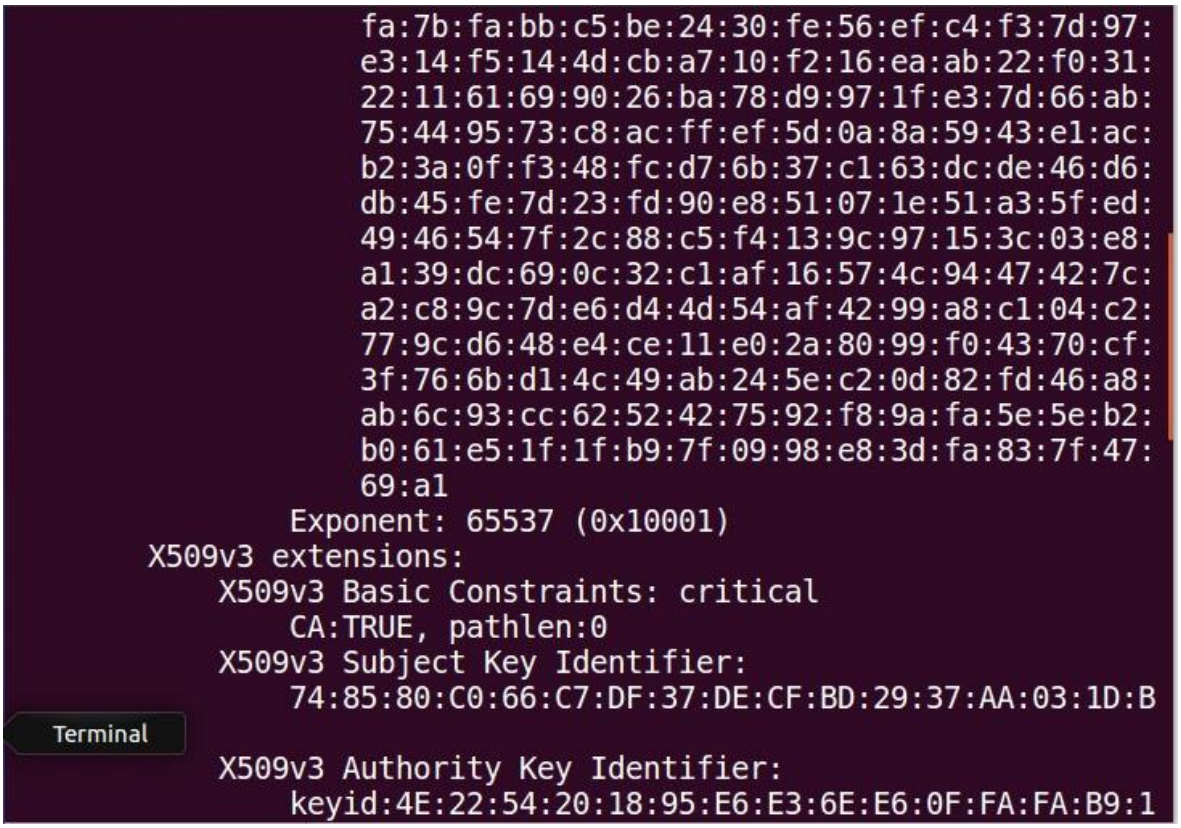
Δραστηριότητα 6: Μη αυτόματη επαλήθευση πιστοποιητικού X.509



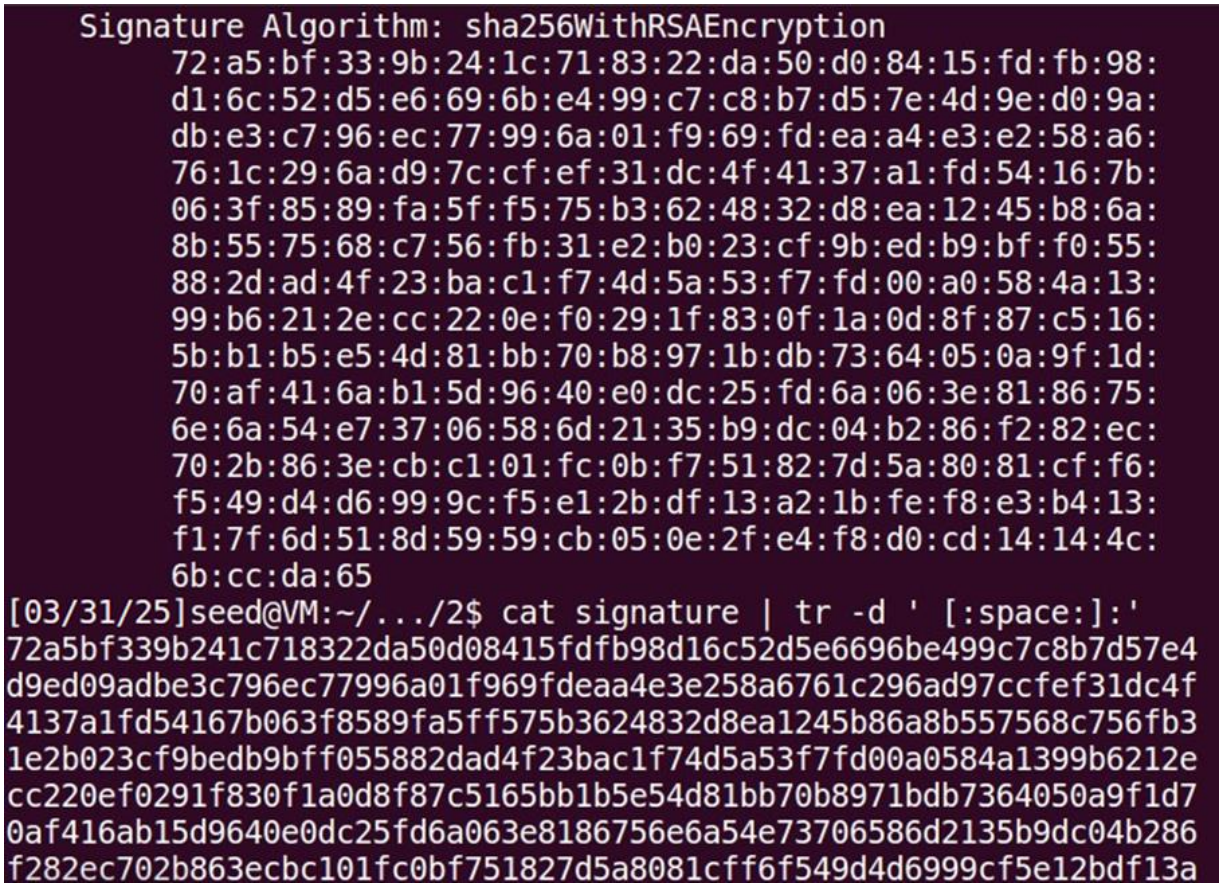
Εικόνα 19: Λήψη του πιστοποιητικού.



Εικόνα 20: Εξαγωγή δημόσιου κλειδιού (N).



Εικόνα 21: Εξαγωγή δημόσιου κλειδιού (e).



Εικόνα 22: Εξαγωγή ψηφιακής υπογραφής.

```
5]seed@VM:~/.../2$ openssl asn1parse -i -in c0.pem
    0:d=0  hl=4 l=1849 cons: SEQUENCE
    4:d=1  hl=4 l=1569 cons:  SEQUENCE
    8:d=2  hl=2 l=   3 cons:   cont [ 0 ]
   10:d=3  hl=2 l=   1 prim:    INTEGER           :02
   13:d=2  hl=2 l=  16 prim:   INTEGER           :063D4917404D39E
513CB3FEECD1B2E1B
   31:d=2  hl=2 l=  13 cons:   SEQUENCE
   33:d=3  hl=2 l=   9 prim:    OBJECT            :sha256WithRSAE
ncryption
   44:d=3  hl=2 l=   0 prim:    NULL
   46:d=2  hl=2 l=  89 cons:   SEQUENCE
   48:d=3  hl=2 l=  11 cons:    SET
   50:d=4  hl=2 l=   9 cons:     SEQUENCE
   52:d=5  hl=2 l=   3 prim:      OBJECT            :countryName
   57:d=5  hl=2 l=   2 prim:      PRINTABLESTRING   :US
   61:d=3  hl=2 l=  21 cons:    SET
   63:d=4  hl=2 l=  19 cons:     SEQUENCE
   65:d=5  hl=2 l=   3 prim:      OBJECT            :organization
Name
   70:d=5  hl=2 l=  12 prim:      PRINTABLESTRING   :DigiCert Inc
   84:d=3  hl=2 l=  51 cons:    SET
   86:d=4  hl=2 l=  49 cons:     SEQUENCE
   88:d=5  hl=2 l=   3 prim:      OBJECT            :commonName
   93:d=5  hl=2 l=  42 prim:      PRINTABLESTRING   :DigiCert Glo
bal G2 TLS RSA SHA256 2020 CA1
 Constraints
 1183:d=5  hl=2 l=   1 prim:      BOOLEAN           :255
 1186:d=5  hl=2 l=   2 prim:      OCTET STRING      [HEX DUMP]:30
00
 1190:d=4  hl=4 l= 383 cons:     SEQUENCE
 1194:d=5  hl=2 l=  10 prim:      OBJECT            :CT Precertif
icate SCTs
 1206:d=5  hl=4 l= 367 prim:      OCTET STRING      [HEX DUMP]:04
82016B01690076004E75A3275C9A10C3385B6CD4DF3F52EB1DF0E08E1B8D69C0B
1FA64B1629A39DF0000018E437CF6800000403004730450220 53F339DBB59CC7
4290DC823B902B86E5632E387452C4A91FD7102326E4A4C8F0022100955F4BAEA
DC200D9483B8A934DD92D59CA0BA45AA24287B863207D17B2B5E1F10076007D59
1E12E1782A7B1C61677C5EFDF8D0875C14A04E959EB9032FD90E8C2E79B800000
18E437CF6810000040300473045022 00B1A4B0436A4F9358A6ABAC21E5667E039
6AC047C037796F9604A8DB51D0B94F022100E272B6FBD9CD25036B2E3163D64FD
D8F14B691BC5AC59FD1D5CC8E95879D1866007700E6D2316340778CC1104106D7
71B9CEC1D240F6968486FBBA87321DFD1E378E500000018E437CF694000004030
0483046022100F2505F8400AC50A3334B0A2B3B162E6AA6994F25321284611D93
81EB35010C90022100D98DD584FE511BE75AA5C6F062055BAD39605B33BB284FE
5835C75D4255CCF74
 1577:d=1  hl=2 l=  13 cons:  SEQUENCE
 1579:d=2  hl=2 l=   9 prim:   OBJECT           :sha256WithRSAEn
cryption
 1590:d=2  hl=2 l=   0 prim:   NULL
 1592:d=1  hl=4 l= 257 prim:  BIT STRING
[03/31/25]seed@VM:~/.../2$
```

Εικόνα 23: Εξαγωγή του σώματος του πιστοποιητικού του server.

```
[03/31/25]seed@VM:~/.../2$ openssl asn1parse -in c0.pem -strparse
 4 -out c0_body.bin -noout
[03/31/25]seed@VM:~/.../2$ sha256sum c0_body.bin
45c12019e2ec23b51760f8b7cd466aea7b5cca737fa6c62e894c33b407387c6e
 c0_body.bin
[03/31/25]seed@VM:~/.../2$
```

Εικόνα 24: Εύρεση και υπολογισμός του hash.

verify_sign_github.c

```c
#include <stdio.h>
#include <openssl/bn.h>

#define NBITS 256

void printBN(char *msg, BIGNUM * a){

    char * number_str = BN_bn2hex(a);
    printf("%s %s\n", msg, number_str);
    OPENSSL_free(number_str);
}

int main(){

    BN_CTX *ctx = BN_CTX_new();
    BIGNUM *n = BN_new();
    BIGNUM *e = BN_new();
    BIGNUM *s = BN_new();
    BIGNUM *m = BN_new();
    BN_hex2bn(&s, "72a5bf339b241c718322da50d08415fdfb98d16c52d5e6696be499c7c8b7d57e
    BN_hex2bn(&e, "10001");
    BN_hex2bn(&n, "CCF710624FA6BB636FED905256C56D277B7A12568AF1F4F9D6E7E18FBD95ABF2

    BN_mod_exp(m, s, e, n, ctx);

    printBN("(*) Verify Sign = ", m);
}
```

Εικόνα 25: Κώδικας για την επαλήθευση του πιστοποιητικού.

```
[03/31/25]seed@VM:.../files$ gcc -o verify_sign_github verify_sig
n_github.c -lcrypto
[03/31/25]seed@VM:.../files$ ./verify_sign_github
(*) Verify Sign =  01FFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFF
FFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFF
FFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFF
FFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFF
FFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFF
FFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFF
FFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFF003031300D060960864801650304020
1050004 2045C12019E2EC23B51760F8B7CD466AEA7B5CCA737FA6C62E894C33B
407387C6E
[03/31/25]seed@VM:.../files$ sha256sum c0_body.bin
45c12019e2ec23b51760f8b7cd466aea7b5cca737fa6c62e894c33b407387c6e
 c0_body.bin
```

Εικόνα 26: Μεταγλώττιση και εκτέλεση του προγράμματος.

Όπως μπορείτε να παρατηρήσετε, η εκτύπωση του προγράμματος, εμφανίζει ένα string του οποίου τα τελευταία ψηφία είναι πανομοιότυπα με αυτά από την παρακάτω εντολή (hash) στην εικόνα. Όλα τα υπόλοιπα μπροστά από αυτό είναι padding που κάνει η hash για να έχουν το ίδιο μήκος.

Εκτυπώσεις προγραμμάτων.

(*) N = E103ABD94892E3E74AFD724BF28E78366D9676BCCC70118BD0AA1968DBB143D1
(*) F(N) = E103ABD94892E3E74AFD724BF28E78348D52298BD687C44DEB3A81065A7981A4
(*) d = 3587A24598E5F2A21DB007D89D18CC50ABA5075BA19A33890FE7C28A9B496AEB
(*) Encrypted Message =
617AB29ECDA432F9B7C5F8E8044404EBFD8B285A699ABED2575B252F78057164
(*) Decrypted Message = 4E696B6F6C616F732054686F6D6173
Name to hex: 4e696b6f6c616f732054686f6d6173
Message : thomasnikos.github.io
(*) Sign = D266BF230BBF87028FF03F2D8722367EA8892E516B0614FF93300CDE13161FB8
Message : thomasnikos.github.ip
(*) Sign (2) = BB26CDB33FCE6C7EF7F5F80D1CD955C52E8671FC0158708F58491594579F26D7

Πιστοποιητικά.

-----BEGIN CERTIFICATE-----
MIIHOTCCBiGgAwIBAgIQBj1JF0BNOeUTyz/uzRsuGzANBgkqhkiG9w0BAQsFADBZ
MQswCQYDVQQGEwJVUzEVMBMGA1UEChMMRGlnaUNlcnQgSW5jMTMwMQYDVQQDEypE
aWdpQ2VydCBHbG9iYWwgRzIgVExTIFJTQSBTSEEyNTYgMjAyMCBDQTEwHhcNMjQw
MzE1MDAwMDAwWhcNMjUwMzE0MjM1OTU5WjBnMQswCQYDVQQGEwJVUzETMBEGA1UE
CBMKQ2FsaWZvcm5pYTEWMBQGA1UEBxMNU2FuIEZyYW5jaXNjbzEVMBMGA1UEChMM
R2l0SHViLCBJbmMuMRQwEgYDVQQDDAsqLmdpdGh1Yi5pbzCCASIwDQYJKoZIhvcN
AQEBBQADggEPADCCAQoCggEBAK0rFKU6TEGvuLCY3ZOuXlG+3jerD6EP1gc1qe35
g68FqyGuVPOUddYNZiymjYMZxywoNp3qxlbFFBTf9etsayavT+uW+2UMjqCotAdK
KicBEspuExoACFuNgTi7sSUT7A55+k4/+5O+VtpaxQ5dmQk7HxcqvMYx5owBU+fB
wYDD+hXeg3YvxLZNeIlN8OlqWL8w9HbG+3ccegVEjOJQbkrcrW7IQMq2Uk92XjxI
PmMVIvaefqcC1poGYvS4VvEh3x64vJK1hEM4YLMKBaE/hqFtcMozi+H/8JqTCfzP
Qhnu21HIop9rSucxxnZbe9AeHz2LERpUTf3rjgOMg9PB1RUCAwEAAaOCA+0wggPp
MB8GA1UdIwQYMBaAFHSFgMBmx9833s+9KTeqAx2+7c0XMB0GA1UdDgQWBBTob1fr
hlGY65+lvlPa25SsKC777TB7BgNVHREEdDByggsqLmdpdGh1Yi5pb4IJZ2l0aHVi
LmlvghVnaXRodWJ1c2VyY29udGVudC5jb22CDnd3dy5naXRodWIuY29tggwqLmdp
dGh1Yi5jb22CFyouZ2l0aHVidXNlcmNvbnRlbnQuY29tggpnaXRodWIuY29tMD4G
A1UdIAQ3MDUwMwYGZ4EMAQICMCkwJwYIKwYBBQUHAgEWG2h0dHA6Ly93d3cuZGln
aWNlcnQuY29tL0NQUzAOBgNVHQ8BAf8EBAMCBaAwHQYDVR0lBBYwFAYIKwYBBQUH
AwEGCCsGAQUFBwMCMIGfBgNVHR8EgZcwgZQwSKBGoESGQmh0dHA6Ly9jcmwzLmRp
Z2ljZXJ0LmNvbS9EaWdpQ2VydEdsb2JhbEcyVExTUlNBU0hBMjU2MjAyMENBMS0x
LmNybDBIoEagRIZCaHR0cDovL2NybDQuZGlnaWNlcnQuY29tL0RpZ2lDZXJ0R2xv
YmFsRzJUTFNSU0FTSEEyNTYyMDIwQ0ExLTEuY3JsMIGHBggrBgEFBQcBAQR7MHkw
JAYIKwYBBQUHMAGGGGh0dHA6Ly9vY3NwLmRpZ2ljZXJ0LmNvbTBRBggrBgEFBQcw
AoZFaHR0cDovL2NhY2VydHMuZGlnaWNlcnQuY29tL0RpZ2lDZXJ0R2xvYmFsRzJU
TFNSU0FTSEEyNTYyMDIwQ0ExLTEuY3J0MAwGA1UdEwEB/wQCMAAwggF/BgorBgEE
AdZ5AgQCBIIBbwSCAWsBaQB2AE51oydcmhDDOFts1N8/Uusd8OCOG41pwLH6ZLFi
mjnfAAABjkN89oAAAAQDAEcwRQIgU/M527Wcx0KQ3II7kCuG5WMuOHRSxKkf1xAj
JuSkyPACIQCVX0uurcIA2Ug7ipNN2S1ZygukWqJCh7hjIH0XsrXh8QB2AH1ZHhLh

eCp7HGFnfF79+NCHXBSgTpWeuQMv2Q6MLnm4AAABjkN89oEAAAQDAEcwRQIgCxpL
BDak+TWKarrCHlZn4DlqwEfAN3lvlgSo21HQuU8CIQDicrb72c0lA2suMWPWT92P
FLaRvFrFn9HVzI6Vh50YZgB3AObSMWNAd4zBEEEG13G5zsHSQPaWhIb7uocyHf0e
N45QAAABjkN89pQAAAQDAEgwRgIhAPJQX4QArFCjM0sKKzsWLmqmmU8lMhKEYR2T
ges1AQyQAiEA2Y3VhP5RG+dapcbwYgVbrTlgWzO7KE/lg1x11CVcz3QwDQYJKoZI
hvcNAQELBQADggEBAHKlvzObJBxxgyLaUNCEFf37mNFsUtXmaWvkmcfIt9V+TZ7Q
mtvjx5bsd5lqAflp/eqk4+JYpnYcKWrZfM/vMdxPQTeh/VQWewY/hYn6X/V1s2JI
MtjqEkW4aotVdWjHVvsx4rAjz5vtub/wVYgtrU8jusH3TVpT9/0AoFhKE5m2IS7M
Ig7wKR+DDxoNj4fFFluxteVNgbtwuJcb23NkBQqfHXCvQWqxXZZA4Nwl/WoGPoGG
dW5qVOc3BlhtITW53ASyhvKC7HArhj7LwQH8C/dRgn1agIHP9vVJ1NaZnPXhK98T
ohv++OO0E/F/bVGNWVnLBQ4v5PjQzRQUTGvM2mU=
-----END CERTIFICATE-----

-----BEGIN CERTIFICATE-----
MIIEyDCCA7CgAwIBAgIQDPW9BitWAvR6uFAsI8zwZjANBgkqhkiG9w0BAQsFADBh
MQswCQYDVQQGEwJVUzEVMBMGA1UEChMMRGlnaUNlcnQgSW5jMRkwFwYDVQQLExB3
d3cuZGlnaWNlcnQuY29tMSAwHgYDVQQDExdEaWdpQ2VydCBHbG9iYWwgUm9vdCBH
MjAeFw0yMTAzMzAwMDAwMDBaFw0zMTAzMjkyMzU5NTlaMFkxCzAJBgNVBAYTAlVT
MRUwEwYDVQQKEwxEaWdpQ2VydCBJbmMxMzAxBgNVBAMTKkRpZ2lDZXJ0IEdsb2Jh
bCBHMiBUTFMgUlNBIFNIQTI1NiAyMDIwIENBMTCCASIwDQYJKoZIhvcNAQEBBQAD
ggEPADCCAQoCggEBAMz3EGJPprtjb+2QUlbFbSd7ehJWivH0+dbn4Y+9lavyYEEV
cNsSAPonCrVXOFt9slGTcZUOakGUWzUb+nv6u8W+JDD+Vu/E832X4xT1FE3LpxDy
FuqrIvAxIhFhaZAmunjZlx/jfWardUSVc8is/+9dCopZQ+GssjoP80j812s3wWPc
3kbW20X+fSP9kOhRBx5Ro1/tSUZUfyyIxfQTnJcVPAPooTncaQwywa8WV0yUR0J8
osicfebUTVSvQpmowQTCd5zWSOTOEeAqgJnwQ3DPP3Zr0UxJqyRewg2C/Uaoq2yT
zGJSQnWS+Jr6Xl6ysGHlHx+5fwmY6D36g39HaaECAwEAAaOCAYIwggF+MBIGA1Ud
EwEB/wQIMAYBAf8CAQAwHQYDVR0OBBYEFHSFgMBmx9833s+9KTeqAx2+7c0XMB8G
A1UdIwQYMBaAFE4iVCAYlebjbuYP+vq5Eu0GF485MA4GA1UdDwEB/wQEAwIBhjAd
BgNVHSUEFjAUBggrBgEFBQcDAQYIKwYBBQUHAwIwdgYIKwYBBQUHAQEEajBoMCQG
CCsGAQUFBzABhhhodHRwOi8vb2NzcC5kaWdpY2VydC5jb20wQAYIKwYBBQUHMAKG
NGh0dHA6Ly9jYWNlcnRzLmRpZ2ljZXJ0LmNvbS9EaWdpQ2VydEdsb2JhbFJvb3RH
Mi5jcnQwQgYDVR0fBDswOTA3oDWgM4YxaHR0cDovL2NybDMuZGlnaWNlcnQuY29t
L0RpZ2lDZXJ0R2xvYmFsUm9vdEcyLmNybDA9BgNVHSAENjA0MAsGCWCGSAGG/WwC
ATAHBgVngQwBATAIBgZngQwBAgEwCAYGZ4EMAQICMAgGBmeBDAECAzANBgkqhkiG
9w0BAQsFAAOCAQEAkPFwyyiXaZd8dP3A+iZ7U6utzWX9upwGnIrXWkOH7U1MVl+t
wcW1BSAuWdH/SvWgKtiwla3JLko716f2b4gp/DA/JIS7w7d7kwcsr4drdjPtAFVS
slme5LnQ89/nD/7d+MS5EHKBCQRfz5eeLjJ1js+aWNJXMX43AYGyZm0pGrFmCW3R
bpD0ufovARTFXFZkAdl9h6g4U5+LXUZtXMYnhIHUfoyMo5tS58aI7Dd8KvvwVVo4
chDYABPPTHPbqjc1qCmBaZx2vN4Ye5DUys/vZwP9BFohFrH/6j/f3IL16/RZkiMN
JCqVJUzKoZHm1Lesh3Sz8W2jmdv51b2EQJ8HmA==
-----END CERTIFICATE-----

Αρχεία εργασίας:

```c
//private_key.c
#include <stdio.h>
#include <openssl/bn.h>

#define NBITS 256

void printBN(char *msg, BIGNUM * a){

    char * number_str = BN_bn2hex(a);
    printf("%s %s\n", msg, number_str);
    OPENSSL_free(number_str);
}

int main(){
    BN_CTX *ctx = BN_CTX_new();
    BIGNUM *p = BN_new();
    BIGNUM *q = BN_new();
    BIGNUM *e = BN_new();
    BIGNUM *t = BN_new();
    BIGNUM *n = BN_new();
    BIGNUM *subp1 = BN_new();
    BIGNUM *subq1 = BN_new();
    BIGNUM *fn = BN_new();
    BIGNUM *d = BN_new();

    BN_hex2bn(&p, "F7E75FDC469067FFDC4E847C51F452DF");
    BN_hex2bn(&q, "E85CED54AF57E53E092113E62F436F4F");
    BN_hex2bn(&e, "0D88C3");
    BN_hex2bn(&t, "1");

    BN_mul(n, p, q, ctx);
    BN_sub(subp1, p, t);
    BN_sub(subq1, q, t);
    BN_mul(fn, subp1, subq1, ctx);
    BN_mod_inverse(d, e, fn, ctx);

    printBN("(*) N = ", n);
    printBN("(*) F(N) = ", fn);
    printBN("(*) d = ", d);

}
```

```c
//encryption.c
#include <stdio.h>
#include <openssl/bn.h>

#define NBITS 256

void printBN(char *msg, BIGNUM * a){

    char * number_str = BN_bn2hex(a);
    printf("%s %s\n", msg, number_str);
    OPENSSL_free(number_str);
}

int main(){

    BN_CTX *ctx = BN_CTX_new();
    BIGNUM *name = BN_new();
    BIGNUM *e = BN_new();
    BIGNUM *n = BN_new();
    BIGNUM *c = BN_new();
    BN_hex2bn(&name, "4E696B6F6C616F732054686F6D6173");
    BN_hex2bn(&e, "0D88C3");
    BN_hex2bn(&n,
"E103ABD94892E3E74AFD724BF28E78366D9676BCCC70118BD0AA1968DBB143D1");

    BN_mod_exp(c, name, e, n, ctx);

    printBN("(*) Encrypted Message = ", c);
}
```

```c
//decryption.c
#include <stdio.h>
#include <openssl/bn.h>

#define NBITS 256

void printBN(char *msg, BIGNUM * a){

    char * number_str = BN_bn2hex(a);
    printf("%s %s\n", msg, number_str);
    OPENSSL_free(number_str);
}

int main(){

    BN_CTX *ctx = BN_CTX_new();
    BIGNUM *d = BN_new();
    BIGNUM *n = BN_new();
    BIGNUM *c = BN_new();
    BIGNUM *m = BN_new();
    BIGNUM *start_m = BN_new();
    BN_hex2bn(&d,
"3587A24598E5F2A21DB007D89D18CC50ABA5075BA19A33890FE7C28A9B496AEB");
    BN_hex2bn(&c,
"617AB29ECDA432F9B7C5F8E8044404EBFD8B285A699ABED2575B252F78057164");
    BN_hex2bn(&n,
"E103ABD94892E3E74AFD724BF28E78366D9676BCCC70118BD0AA1968DBB143D1");
    BN_hex2bn(&start_m, "4E696B6F6C616F732054686F6D6173");

    BN_mod_exp(m, c, d, n, ctx);

    printBN("(*) Decrypted Message = ", m);
    if(BN_cmp(start_m,m) == 0){
        printf("Messages are the same!\n");
    }else{
        printf("Messages are not the same!\n");
    }
}
```

```c
//sign.c
#include <stdio.h>
#include <openssl/bn.h>

#define NBITS 256

void printBN(char *msg, BIGNUM * a){

    char * number_str = BN_bn2hex(a);
    printf("%s %s\n", msg, number_str);
    OPENSSL_free(number_str);
}

int main(){

    BN_CTX *ctx = BN_CTX_new();
    BIGNUM *d = BN_new();
    BIGNUM *n = BN_new();
    BIGNUM *m = BN_new();
    BIGNUM *s = BN_new();
    BN_hex2bn(&d,
"3587A24598E5F2A21DB007D89D18CC50ABA5075BA19A33890FE7C28A9B496AEB");
    BN_hex2bn(&m, "74686F6D61736E696B6F732E6769746875622E696F");
    BN_hex2bn(&n,
"E103ABD94892E3E74AFD724BF28E78366D9676BCCC70118BD0AA1968DBB143D1");

    BN_mod_exp(s, m, d, n, ctx);

    printBN("(*) Sign = ", s);
}
```

```c
//verify_sign_alice.c
#include <stdio.h>
#include <openssl/bn.h>

#define NBITS 256

void printBN(char *msg, BIGNUM * a){

    char * number_str = BN_bn2hex(a);
    printf("%s %s\n", msg, number_str);
    OPENSSL_free(number_str);
}

int main(){

    BN_CTX *ctx = BN_CTX_new();
    BIGNUM *n = BN_new();
    BIGNUM *e = BN_new();
    BIGNUM *s = BN_new();
    BIGNUM *m = BN_new();
    BIGNUM *start_m = BN_new();
    BN_hex2bn(&s,
"643D6F34902D9C7EC90CB0B2BCA36C47FA37165C0005CAB026C0542CBDB6803F");
    BN_hex2bn(&e, "010001");
    BN_hex2bn(&n,
"AE1CD4DC432798D933779FBD46C6E1247F0CF1233595113AA51B450F18116115");
    BN_hex2bn(&start_m, "4C61756E63682061206D697373696C652E");
    BN_mod_exp(m, s, e, n, ctx);

    printBN("(*) Verify Sign = ", m);
    if(BN_cmp(start_m,m) == 0){
        printf("Messages are the same!\n");
    }else{
        printf("Messages are not the same!\n");
    }
}
```

```c
//verify_sign_bob.c
#include <stdio.h>
#include <openssl/bn.h>

#define NBITS 256

void printBN(char *msg, BIGNUM * a){

    char * number_str = BN_bn2hex(a);
    printf("%s %s\n", msg, number_str);
    OPENSSL_free(number_str);
}

int main(){

    BN_CTX *ctx = BN_CTX_new();
    BIGNUM *n = BN_new();
    BIGNUM *e = BN_new();
    BIGNUM *s = BN_new();
    BIGNUM *m = BN_new();
    BIGNUM *start_m = BN_new();

    BN_hex2bn(&s,
"DB3F7CDB93483FC1E70E4EACA650E3C6505A3E5F49EA6EDF3E95E9A7C6C7A320");
    BN_hex2bn(&e, "010001");
    BN_hex2bn(&n,
"DCBFFE3E51F62E09CE7032E2677A78946A849DC4CDDE3A4D0CB81629242FB1A5");
    BN_hex2bn(&start_m,
"506C65617365207472616E73666572206D652024323030302E416C6963652E");

    BN_mod_exp(m, s, e, n, ctx);

    printBN("(*) Verify Sign = ", m);
    if(BN_cmp(start_m,m) == 0){
        printf("Messages are the same!\n");
    }else{
        printf("Messages are not the same!\n");
    }
}
```

```c
#include <stdio.h>
#include <openssl/bn.h>

#define NBITS 256

void printBN(char *msg, BIGNUM * a){

    char * number_str = BN_bn2hex(a);
    printf("%s %s\n", msg, number_str);
    OPENSSL_free(number_str);
}

int main(){

    BN_CTX *ctx = BN_CTX_new();
    BIGNUM *n = BN_new();
    BIGNUM *e = BN_new();
    BIGNUM *s = BN_new();
    BIGNUM *m = BN_new();
    BN_hex2bn(&s,
"45c12019e2ec23b51760f8b7cd466aea7b5cca737fa6c62e894c33b407387c6e");
    BN_hex2bn(&e, "10001");
    BN_hex2bn(&n,
"CCF710624FA6BB636FED905256C56D277B7A12568AF1F4F9D6E7E18FBD95ABF2604115
70DB1200FA270AB557385B7DB2519371950E6A41945B351BFA7BFABBC5BE2430FE56EFC
4F37D97E314F5144DCBA710F216EAAB22F031221161699026BA78D9971FE37D66AB7544
9573C8ACFFEF5D0A8A5943E1ACB23A0FF348FCD76B37C163DCDE46D6DB45FE7D23FD90E
851071E51A35FED4946547F2C88C5F4139C97153C03E8A139DC690C32C1AF16574C9447
427CA2C89C7DE6D44D54AF4299A8C104C2779CD648E4CE11E02A8099F04370CF3F766BD
14C49AB245EC20D82FD46A8AB6C93CC6252427592F89AFA5E5EB2B061E51F1FB97F0998
E83DFA837F4769A1");

    BN_mod_exp(m, s, e, n, ctx);

    printBN("(*) Verify Sign = ", m);
}
```